

# TUI Editor Widget Comparison

Research for task `research-embeddable-rust` — evaluating embeddable Rust TUI editor widgets to replace the custom text input implementation.

## Current Implementation Analysis

The TUI currently uses a **fully custom text editing system** spread across `src/tui/viz_viewer/event.rs` and `state.rs`:

- **handle\_text\_editing()** (`event.rs:613-750`): Shared function handling character insertion, deletion, Emacs keybindings (`Ctrl+A/E/B/F/K/U/W/Y/D`), arrow keys, Home/End, Up/Down for multiline navigation, and a kill ring.
- **TextPromptState** (`state.rs:1024-1030`): `Simple { input: String, cursor: usize, scroll: usize }`.
- **ChatState** uses similar fields: `input`, `cursor`, `input_scroll`.
- **InputMode enum** (`state.rs:427-441`): Tracks focus via modes — `Normal`, `ChatInput`, `MessageInput`, `TextPrompt(action)`, `TaskForm`, `Search`, `ConfigEdit`, `Confirm`.
- **Rendering** (`render.rs:1532+`): Custom `build_input_visual_lines()` handles word wrapping, cursor positioning, and viewport scrolling.

## Current Problems (from task description)

1. **Focus leak** — captures keystrokes when not focused
2. **Limited cursor navigation** — has up/down/left/right but no word-jump, no selection
3. **Basic scrolling** — auto-scroll only, no scroll bar or mouse scroll in input
4. **No mouse support** — can't click to position cursor within input area

## What Already Works Well

- Emacs keybindings (`Ctrl+A/E/B/F/K/U/W/Y`)
- Kill ring (cut/paste buffer)
- Multiline editing with `Shift+Enter` / `Alt+Enter`
- Up/Down cursor movement across lines
- Auto-scroll viewport to cursor

---

## Candidate Comparison

### 1. tui-textarea (rhysd/tui-textarea)

**Already a dependency** — `Cargo.toml` line 55: `tui-textarea = { version = "0.7", features = ["crossterm"] }` (currently unused).

| Criterion                | Assessment                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>ratatui compat</b>    | Native support. <code>&amp;TextArea</code> implements <code>Widget</code> . Uses <code>ratatui</code> types directly. |
| <b>Multiline editing</b> | Full multiline with auto-scrolling, line numbers, cursor line highlight.                                              |
| <b>Cursor movement</b>   | Arrow keys, word-jump ( <code>Alt+F/B</code> ), Home/End, <code>Ctrl+A/E</code> , page up/down.                       |
| <b>Scrolling</b>         | Auto-scroll on cursor move. Page scroll ( <code>Ctrl+V/Alt+V</code> , <code>PageDown/PageUp</code> ).                 |

| Criterion                 | Assessment                                                                                                                                                                                                                                                                                                               |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mouse support</b>      | Mouse scroll support. No click-to-position.                                                                                                                                                                                                                                                                              |
| <b>Clipboard</b>          | Internal yank buffer (Ctrl+K/J). Copy/Cut/Paste (Ctrl+C/X/Y). No system clipboard.                                                                                                                                                                                                                                       |
| <b>Text selection</b>     | Supported.                                                                                                                                                                                                                                                                                                               |
| <b>Undo/Redo</b>          | Built-in (Ctrl+U/R).                                                                                                                                                                                                                                                                                                     |
| <b>Search</b>             | Optional regex search (search feature).                                                                                                                                                                                                                                                                                  |
| <b>Focus management</b>   | Explicit — app calls <code>textarea.input(event)</code> only when focused. Widget is stateful; rendering is via <code>&amp;textarea</code> reference. Focus is entirely app-controlled.                                                                                                                                  |
| <b>Maturity</b>           | 381 commits. v0.7. Active. 36 issues, 16 PRs. Widely used.                                                                                                                                                                                                                                                               |
| <b>Integration effort</b> | <b>Low</b> — already a dependency. Replace <code>TextPromptState</code> and chat input <code>String+cursor</code> with <code>TextArea</code> instances. Route events via <code>textarea.input()</code> only in the matching <code>InputMode</code> . Render with <code>frame.render_widget(&amp;textarea, area)</code> . |

#### Key API pattern:

```

let mut textarea = TextArea::default();
// Only pass input when focused:
if focused {
    textarea.input(crossterm_event);
}
// Render:
frame.render_widget(&textarea, area);
// Get content:
let lines: &[String] = textarea.lines();

```

**Pros:** - Already in Cargo.toml — zero new dependencies - Explicit input routing = clean focus management - Covers all our needs: multiline, cursor nav, scrolling, undo/redo - Well-tested, widely adopted in the ratatui ecosystem - `input_without_shortcuts()` allows custom key handling while keeping basic editing

**Cons:** - No click-to-position cursor (mouse scroll only) - No system clipboard (internal yank only) — but we already have this limitation - Emacs-style defaults; would need to verify our existing keybindings map cleanly

## 2. edtui (preiter93/edtui)

| Criterion                | Assessment                                                       |
|--------------------------|------------------------------------------------------------------|
| <b>ratatui compat</b>    | Native. <code>EditorView</code> implements <code>Widget</code> . |
| <b>Multiline editing</b> | Full multiline with line wrapping.                               |
| <b>Cursor movement</b>   | Vim-style (hjkl, w/b/e, 0/\$, gg/G) or Emacs mode.               |
| <b>Scrolling</b>         | Viewport scrolling, half-page (Ctrl+D/U in Vim mode).            |
| <b>Mouse support</b>     | Full — click to position cursor, mouse event handling built-in.  |

| Criterion                 | Assessment                                                                                                                                                                                              |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clipboard</b>          | Paste via bracketed paste. System editor integration (Ctrl+E opens nvim).                                                                                                                               |
| <b>Text selection</b>     | Visual mode (Vim) or shift-select (Emacs).                                                                                                                                                              |
| <b>Undo/Redo</b>          | Built-in.                                                                                                                                                                                               |
| <b>Search</b>             | Not mentioned in docs.                                                                                                                                                                                  |
| <b>Focus management</b>   | Event-driven — app calls <code>handler.on_key_event()</code> / <code>handler.on_event()</code> . Focus is app-controlled.                                                                               |
| <b>Maturity</b>           | 341 commits. v0.11.1 (52K downloads). 8 open issues. Active (last update Feb 2026).                                                                                                                     |
| <b>Integration effort</b> | <b>Medium</b> — new dependency. More complex API (EditorState + EditorView + EditorEventHandler). Vim modal editing may confuse users expecting simple text input. Would need Emacs mode configuration. |

#### Key API pattern:

```
let mut state = EditorState::new(vec!["initial text".into()]);
let handler = EditorEventHandler::default(); // Vim mode
// or: EditorEventHandler::new(KeyEventHandler::emacs());
handler.on_key_event(&mut state, key_event);
// Render:
EditorView::new(&mut state).theme(theme).render(area, buf);
```

**Pros:** - Best mouse support (click to position cursor) - Vim and Emacs modes - Syntax highlighting support (unnecessary for our use case) - System editor integration (open in nvim) - Active maintenance

**Cons:** - New dependency (not already in Cargo.toml) - More complex API — three separate types to manage - Vim modal editing is overkill for chat/message input boxes - Heavier — syntax highlighting, line numbers, themes are features we mostly don't need - Less established than tui-textarea (52K vs much higher downloads)

### 3. tui-input / ratatui\_input

| Criterion                 | Assessment                                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| <b>ratatui compat</b>     | Yes.                                                                                                  |
| <b>Multiline editing</b>  | <b>No</b> — single-line only.                                                                         |
| <b>Scrolling</b>          | Horizontal scrolling/windowing for long single-line input.                                            |
| <b>Mouse support</b>      | Unknown/limited.                                                                                      |
| <b>Focus management</b>   | Event-driven, app-controlled.                                                                         |
| <b>Maturity</b>           | ratatui_input: v0.1, "under heavy construction and not ready for use". tui-input: basic, single-line. |
| <b>Integration effort</b> | N/A — doesn't meet multiline requirement.                                                             |

**Verdict:** Eliminated — no multiline support, immature.

#### 4. Custom implementation (status quo, improved)

| Criterion                 | Assessment                                                                                                                 |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Integration effort</b> | Zero — it's what we have.                                                                                                  |
| <b>Focus management</b>   | Already works via <code>InputMode</code> enum.                                                                             |
| <b>Feature gaps</b>       | Missing: text selection, undo/redo, word-jump, mouse click-to-position.                                                    |
| <b>Maintenance</b>        | All bugs and features are our responsibility. ~140 lines of <code>handle_text_editing</code> + rendering code to maintain. |

**Verdict:** Not recommended — we'd be reimplementing what `tui-textarea` already provides.

#### Recommendation: `tui-textarea`

`tui-textarea` is the clear winner for our use case:

1. **Already a dependency** — it's in `Cargo.toml` v0.7 with `crossterm` features, just unused. Zero additional dependency cost.
2. **Perfect scope** — it's a text input widget, not a full editor. This matches our use case (chat input, message input, description editing).
3. **Clean focus model** — `textarea.input(event)` is only called when we want it to receive input. Our `InputMode` enum maps directly to this pattern.
4. **Feature uplift** — immediately gains undo/redo, text selection, better scrolling, and search that our custom code lacks.
5. **Less code to maintain** — replaces ~200 lines of custom editing + rendering code with a well-tested library.

`edtui` would be the choice if we needed a full code editor (vim bindings, syntax highlighting, click-to-position). But for text input boxes, `tui-textarea` is simpler and already available.

#### Focus Management Patterns (applicable regardless of choice)

Regardless of which editor widget we use, these patterns should be adopted:

##### 1. Guard all input routing with mode checks

```
// Current pattern (good – keep this):
match app.input_mode {
    InputMode::ChatInput => { /* route to editor */ }
    InputMode::Normal => { /* route to app navigation */ }
    ...
}
```

##### 2. Never let the widget see events when unfocused

The fix for “focus leak” is straightforward with `tui-textarea`: only call `textarea.input()` inside the matching `InputMode` branch. The widget has no global event listener — it only processes what you give it.

### 3. Use `input_without_shortcuts()` for custom override keys

`tui-textarea`'s `input_without_shortcuts()` processes only basic character insertion/deletion, letting us handle `Esc`, `Enter`, `Ctrl+C` etc. ourselves:

```
match key.code {
  KeyCode::Esc => { /* exit input mode */ }
  KeyCode::Enter => { /* submit */ }
  _ => { textarea.input(key_event); }
}
```

### 4. Separate widget state from app state

Currently `TextPromptState` holds `{ input: String, cursor: usize, scroll: usize }`. With `tui-textarea`, replace this with a `TextArea` instance that owns all its state internally. Extract content with `textarea.lines()` when submitting.

### 5. Click-to-focus pattern

For mouse support, store the rendered `Rect` of each input area (already done for `last_chat_input_area`) and on mouse click, check if the click falls within an input area to switch `InputMode` and focus the corresponding widget.

---

## Integration Sketch

```
// In VizApp state:
pub chat_textarea: TextArea<'static>,
pub message_textarea: TextArea<'static>,
pub prompt_textarea: TextArea<'static>,

// In event handling:
InputMode::ChatInput => {
  match (code, modifiers) {
    (KeyCode::Esc, _) => { app.input_mode = InputMode::Normal; }
    (KeyCode::Enter, KeyModifiers::NONE) => {
      let text = app.chat_textarea.lines().join("\n");
      app.chat_textarea = TextArea::default(); // clear
      app.send_chat_message(text);
    }
    _ => { app.chat_textarea.input(event); }
  }
}
```

```
// In rendering:
frame.render_widget(&app.chat_textarea, input_area);
```

This replaces `handle_text_editing()`, `handle_chat_input()`, `build_input_visual_lines()`, `move_cursor_up/down()`, `line_start/end()`, `prev/next_char_boundary()`, and the manual scroll tracking — all with a single `textarea.input() + render_widget()` pair.